Architectural Divergence and Epistemic Integrity in the Stela Context Engine: A Unified Strategy for Consolidation

1. Introduction: The Epistemic Architecture of Repo-Resident AI

The Stela platform represents a paradigm shift in autonomous software development, moving beyond simple script execution to a model of "repo-resident AI" governed by strict epistemic standards. At the heart of this architecture lies the Policy of Truth (PoT), a constitutional document that establishes the "Physical Laws" of the repository.[1] Unlike traditional development environments where truth is often fragmented across wiki pages, issue trackers, and code comments, Stela mandates a Single Source of Truth (SSOT) rooted directly in the file system. This "Filing Doctrine" segregates the repository into distinct teleological domains: ops/ for execution, docs/ for explanation, and projects/ for payload work.[1]

However, maintaining this high standard of epistemic integrity requires a sophisticated mechanism for "hydrating" AI agents—reconstructing the full operational context from a stateless repository. This is the function of the Stela Context Engine. Ideally, this engine should provide a unified, non-contradictory view of the repository's reality. Yet, a forensic analysis of the current system reveals a critical architectural fracture. The platform has evolved two parallel pipelines for context generation: the execution-focused ops/bin/context and the discovery-focused ops/bin/llms.[1]

This bifurcation has birthed a "Two Truths" operational drift. The reality presented to an active agent (the "Execution Truth") differs structurally and semantically from the reality presented to external indexers or high-level planning agents (the "Discovery Truth") via the llms.txt standard. This divergence is not merely a technical inefficiency; it is a governance failure that forces operators to rely on manual exception handling—specifically within docs/ops/prompts/—to bridge the gap between what the system claims to be true and what the agent can actually see.

This report provides an exhaustive analysis of these subsystems. It dissects the bash-level logic of the binaries, evaluates the redundancy of static context bundles like llms-governance.txt and llms-ops.txt, and scrutinizes the conflicting instructions embedded in operator prompts and linting tools. The ultimate objective is to propose a "One Truth" consolidation strategy that restores the platform's alignment with its own Policy of Truth.

---

2. Forensic Architecture Analysis: The Context Subsystems

To understand the nature of the "Two Truths" drift, one must move beyond high-level descriptions and analyze the executable logic that defines these realities. The repository contains two primary binaries responsible for synthesizing context, each operating under different constraints and manifests.

2.1 The Execution Pipeline: ops/bin/context

The ops/bin/context binary serves as the "Context Archivist." Its primary teleology is **action**. It is designed to prepare a specific, ephemeral snapshot of the repository for an agent tasked with immediate work.[1]

2.1.1 Architectural Mechanics

Forensic examination of the script [1] reveals that ops/bin/context functions primarily as an orchestrator or wrapper. It does not perform the heavy lifting of file selection itself; rather, it delegates this to ops/bin/open and ops/bin/dump.

The binary executes ops/bin/open --out=auto to generate the session header—a structured prompt containing the current git branch, commit hash (HEAD), and "freshness gate" data.[1] This establishes the temporal context of the session. It then executes ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle to serialize the repository's file contents. Finally, it bundles these artifacts (the OPEN header and the DUMP payload) into a single archive file located in storage/archives/context/.

2.1.2 The Manifest Constraint

The critical governance constraint for this pipeline is the input manifest. ops/bin/dump, which ops/bin/context invokes, relies on ops/lib/manifests/CONTEXT.md to determine which files are relevant.[1] The logic within ops/bin/dump parses this markdown file, extracting backticked file paths.

Crucially, this pipeline is subject to the **Context Hazard Axiom** defined in PoT.md: "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1] This axiom is strictly enforced to prevent "instruction drift," where an agent might confuse a hypothetical task description in the library with a live directive in the Dispatch Packet (DP). Therefore, the Execution Truth generated by ops/bin/context is intentionally lobotomized; it contains the *capabilities* of the system (the tools) but excludes the *definitions* of those capabilities (the library).

2.2 The Discovery Pipeline: ops/bin/llms

The ops/bin/llms binary functions as the "Context Bundler." Its teleology is **comprehension**. Unlike the ephemeral nature of context, llms is designed to create static, persistent artifacts (llms.txt, llms-full.txt) that allow external systems or "Architect" agents to explore the repository's layout and capabilities.[1]

2.2.1 Architectural Mechanics

The script ops/bin/llms [1] operates independently of the dump tool. It implements its own manifest parsing logic, reading from ops/lib/manifests/CONTEXT.md (for the "Small" bundle) and ops/lib/manifests/LLMS.md (for specialized bundles like "Ops" and "Governance").

A detailed code review of ops/bin/llms reveals significant logic duplication. It contains functions for:

- **strip_toc**: An awk script to remove Table of Contents blocks, optimizing token usage.[1]
- **redact_stream**: A sed pipeline that strips sensitive patterns (e.g., AKIA..., ghp_...) from the output.[1]
- **emit_sop_head**: Logic to truncate the SoP.md history ledger to the most recent 10 entries.[1]
- **bundle_section_paths**: Logic to extract file paths from specific sections of the manifest.[1]

2.2.2 The Divergence of Truth

The critical divergence occurs in how ops/bin/llms handles the Context Hazard. While ops/bin/context strictly excludes the library, ops/bin/llms *must* include it to serve its purpose as a discovery map. The generated llms-full.txt bundle aggregates content from the Ops and Governance manifests but notably creates a comprehensive view that the execution pipeline lacks.[1]

This creates the "Two Truths" drift. An Architect agent reading llms-full.txt sees a repository rich with agents, skills, and tasks. It drafts a plan assuming these resources are available. However, when that plan is handed to a Contractor agent hydrated via ops/bin/context, the library files are missing. The Contractor is blind to the very tools the Architect instructed it to use, leading to hallucination or failure.

2.3 Logic Duplication and Maintenance Risk

The existence of two separate implementations for manifest parsing and file processing violates the **Reuse-first** axiom of the Policy of Truth.[1]

Table 1: Logic Duplication Analysis

Logic Component	Implementation in	Implementation in

	ops/bin/dump (used by Context)	ops/bin/llms (Discovery)
Manifest Parsing	Uses git ls-files combined with scope filters (platform, full).	Custom awk script to parse markdown backticks from LLMS.md.
Hazard Enforcement	Checks excludes via arguments; relies on tools/lint/context.sh.	Custom validate_bundle_paths function iterating over hazard array.
Redaction	Metadata-only binary handling (--include-binary=meta).	sed based regex replacement for API keys.
Output Formatting	Structured "Dump Payload" with file delimiters.	Concatenated markdown with ## filename headers.

This duplication is a vector for operational drift. If the definition of a "Context Hazard" changes in PoT.md (e.g., adding a new restricted directory), an operator must remember to update the hazard arrays in both ops/bin/dump and ops/bin/llms separately. History shows that manual synchronization is the primary cause of governance failure in complex systems.

---

3. The Bundle Ecosystem: Analysis of Fragmentation

The Stela architecture attempts to manage context window limitations through "Context Slicing"—the creation of specialized text bundles. The repository currently maintains llms.txt, llms-small.txt, llms-full.txt, llms-ops.txt, and llms-governance.txt.[1] While theoretically sound, in practice, this fragmentation introduces cognitive load and redundancy.

3.1 The "Governance" vs. "Ops" False Dichotomy

The separation of llms-governance.txt and llms-ops.txt is based on the Filing Doctrine's distinction between docs/ (Explain) and ops/ (Run).

- **llms-governance.txt** contains PoT.md, SoP.md, TASK.md, and docs/GOVERNANCE.md.[1] It represents the "Legislative" branch of the repository.
- **llms-ops.txt** contains docs/MANUAL.md, docs/ops/**/*.md, and ops/bin/* source code.[1] It represents the "Executive" branch.

Critique: In a repo-resident AI system, legislation and execution are inextricable. The PoT.md (Governance) explicitly references ops/bin/open (Ops) as the mandatory mechanism for session initiation.[1] Conversely, ops/bin/open (Ops) explicitly injects PoT.md (Governance) into the prompt header.[1]

An agent reading only llms-governance.txt knows *that* it must use ops/bin/open but does not know the command-line arguments to do so (which are in docs/MANUAL.md inside llms-ops.txt). An agent reading only llms-ops.txt knows *how* to use the binary but does not understand the "Freshness Gate" constraints imposed by the PoT. This separation forces the Architect to load multiple bundles, negating the efficiency gains of slicing.

3.2 The Redundancy of llms-small.txt

llms-small.txt is defined as the "Small Canonical Bundle," containing PoT.md, SoP.md, and TASK.md.[1] This is identical to the "Canonical Context" defined in ops/lib/manifests/CONTEXT.md.

This redundancy creates a dangerous "Ghost Canon." CONTEXT.md is the *actual* definition used by the system for execution. llms-small.txt is a *static copy* of that definition used for external display. If CONTEXT.md is updated to include a new critical file (e.g., docs/SECURITY.md), llms-small.txt remains stale until ops/bin/llms is manually re-run. This lag creates a window where external observers believe the system has security context, while the actual agents do not.

3.3 The llms-full.txt Aggregation Risk

The ops/bin/llms script constructs llms-full.txt by aggregating the paths from Small, Full, Ops, and Governance sections of the manifests.[1] This means the content of PoT.md is physically written into llms-small.txt, llms-governance.txt, and llms-full.txt.

This triple-storage strategy is highly fragile. Any manual edit to one of these generated files (a violation of protocol, but possible) creates an immediate "Split-Brain" scenario. The tools/lint/llms.sh script [1] exists solely to police this redundancy, regenerating the bundles in a temporary directory and diffing them against the root files. The existence of a linter specifically to check if file A is a copy of file B is a strong code smell indicating architectural inefficiency.

---

4. Operational Exceptions and Prompt Provisions

The friction generated by this fragmented architecture is most visible in the "patches" applied to the governance layer—specifically within the Operator Prompts (docs/ops/prompts/) and

the Dispatch Packet linter (tools/lint/dp.sh).

4.1 The E-PROMPT-03 Manual Override

The **Architect (Refresh + Draft DP)** stance, defined in docs/ops/prompts/E-PROMPT-03.md [1], contains a highly specific manual addendum regarding LLMS operations:

> "**Addendum:** If the DP plan includes an llms command invocation or a context bundle refresh objective, the DP allowlist must include the repository root outputs: llms.txt, llms-small.txt, llms-full.txt, llms-ops.txt, and llms-governance.txt."

This provision is an admission of failure in the system's dependency resolution.

1. **Cognitive Load:** It forces the human or AI operator to manually remember a list of five specific filenames every time they touch the context engine.
2. **Hardcoding:** It hardcodes the names of the bundles into the prompt. If the system evolves to use llms-core.txt instead of llms-small.txt, this prompt becomes "Ghost Canon"—an instruction that is formally valid but functionally incorrect.
3. **Allowlist Abuse:** The Stela allowlist is intended to prevent accidental modification of out-of-scope files. By mandating the inclusion of all generated bundles, the prompt essentially opens write access to the entire root surface of the repository for any context-related task. This violates the **Principle of Least Privilege**.

4.2 The Linter Conflict (tools/lint/dp.sh)

While the prompt mandates the inclusion of these files in the *allowlist* (for writing), the Dispatch Packet linter (tools/lint/dp.sh) contains logic that actively warns against their inclusion in the *context* (for reading).[1]

The script performs a check:

Bash

```
if grep -Ei "llms.*\.txt" "$TARGET"; then
  echo "DP WARNING: Task surface contains llms discovery files. This is usually redundant context."
fi
```

This creates a direct conflict in the "Two Truths."

- **The Architect (Prompt):** "You MUST list these files."
- **The Gatekeeper (Linter):** "You SHOULD NOT include these files."

To an AI agent, this contradiction is paralyzing. It leads to uncertainty about whether "Allowlisted" implies "Read into Context." If the agent reads llms-full.txt (because it was allowlisted) while also reading the raw docs/ (as per CONTEXT.md), it consumes its context window with 100% redundant token usage, often truncating the actual instructions at the end of the prompt.

---

5. Historical Teleology: The Evolution of Drift

To understand how Stela arrived at this state, we must examine the State of Play (SoP.md) ledger [1], which records the system's evolution.

- **DP-OPS-0035 (Context Lifecycle Hardening):** This dispatch packet introduced ops/bin/llms and the scope bundles. It codified the separation of llms-ops.txt and llms-governance.txt. The intent was likely to support smaller context windows of previous-generation models (e.g., GPT-3.5 or early Claude 2).
- **DP-OPS-0057 (Ops Surface Operational Certification):** This update refreshed the llms bundles and updated the manifests. It solidified the "static text file" approach as the standard for discovery.

The drift is a legacy artifact. The architecture was optimized for a constraint (small context windows) that is rapidly becoming obsolete. The maintenance cost of the "sliced" bundles now outweighs the token savings, especially given the "Two Truths" consistency errors they introduce.

---

6. Proposed Strategy: The "One Truth" Architecture

To resolve these operational schisms and restore strict adherence to the Policy of Truth, Stela must adopt a unified strategy for context generation. This strategy consolidates manifests, unifies binary logic, and deprecates redundant artifacts.

6.1 Unified Manifest Core (The Hierarchy of Truth)

The current parallel manifests (CONTEXT.md vs LLMS.md) must be replaced by a hierarchical inheritance model rooted in a single immutable core.

Table 2: Proposed Unified Manifest Structure

Manifest Level	File Location	Content Scope	Inherits From	Purpose

Layer 0: Core	ops/lib/manifests/CORE.md	PoT.md, SoP.md, TASK.md, docs/MAP.md	(None)	The absolute minimum "Physical Laws" required for *any* agent interaction.
Layer 1: Ops	ops/lib/manifests/OPS.md	docs/MANUAL.md, ops/bin/*, docs/ops/**/*.md	CORE.md	Operational mechanics. Required for agents performing system maintenance.
Layer 2: Discovery	ops/lib/manifests/DISCOVERY.md	README.md, docs/INDEX.md, docs/library/INDEX.md	CORE.md	External indexing. This layer is the *only* one allowed to reference Library indexes.

Operational Impact:

- ops/bin/context (Session) defaults to loading **Layer 1 (Ops)**. This ensures every agent has both the Laws (Governance) and the Tools (Ops).
- ops/bin/llms (Discovery) defaults to loading **Layer 2 (Discovery)**.
- **Drift Elimination:** Because OPS.md and DISCOVERY.md both inherit CORE.md, any change to the constitution (PoT.md) is instantly propagated to both the execution agent and the external index.

6.2 Unified Binary Architecture (ops/bin/synthesize)

To eliminate logic duplication, we propose a new internal binary or library, ops/lib/scripts/synthesize.sh, which centralizes the context generation logic.

Core Functions of synthesize.sh:

1. **resolve_manifest(path):** A robust parser that handles markdown backticks and glob expansion.
2. **enforce_hazards(file_list):** A centralized implementation of the Context Hazard Axiom.

It checks every file against a single blacklist of forbidden directories (docs/library/*).
3. **emit_stream(file_list):** Standardizes the output format (headers, delimiters) for both sessions and static files.

Refactored Binaries:

- **ops/bin/context:** Becomes a lightweight wrapper around synthesize.sh. It injects the OPEN artifact (session state) and streams the result to STDOUT or a tarball.
- **ops/bin/llms:** Becomes a wrapper around synthesize.sh. It calls the synthesizer to generate the file content and writes it to llms.txt and llms-full.txt.

6.3 Deprecation of Redundant Bundles

The strategy mandates the immediate deprecation of llms-governance.txt and llms-ops.txt.

- **New llms.txt Standard:** The repository root will contain only:
  - llms.txt: The index file (compliant with llmstxt.org standard).
  - llms-core.txt: Generated from CORE.md (replaces llms-small.txt).
  - llms-full.txt: Generated from DISCOVERY.md (replaces all other bundles).
- **Justification:** Modern LLMs handle large contexts effectively. The cognitive load of choosing between "Governance" and "Ops" bundles leads to errors. Agents should ingest the CORE for alignment and query the MAP to retrieve specific operational docs as needed, rather than relying on pre-sliced text files.

6.4 Policy and Prompt Hardening

With the architecture unified, the manual overrides in the prompts can be removed.

1. Update E-PROMPT-03:

- **Remove:** The entire "LLMS Addendum" listing specific filenames.
- **Replace with:** "If context refreshment is required, include ops/bin/llms in the allowlist. The system will manage the artifacts."

2. Update tools/lint/dp.sh:

- Refine the warning logic. Instead of a blanket warning on llms*.txt, explicitly **Fail** if llms-full.txt is detected in the context (as it is too large and redundant), but **Allow** llms-core.txt if the agent specifically requests a lightweight alignment check.

7. Conclusion

The fragmentation of the Stela context engine is a classic case of architectural entropy—a system growing more complex than its governing laws can contain. The "Two Truths" drift between context and llms undermines the core promise of the Policy of Truth: that the repository is a consistent, verifiable reality.

By adopting the "One Truth" strategy, Stela aligns its architecture with its axioms. The **Unified Manifest** ensures that every agent, regardless of role, operates on the same constitutional foundation. The **Unified Binary** logic eliminates the risk of divergent hazard enforcement. And the **Deprecation of Redundant Bundles** simplifies the cognitive environment, allowing agents to focus on execution rather than navigating a maze of static text files. This consolidation is not merely a cleanup; it is a re-founding of the platform's epistemic integrity.

8. Appendix: Technical Specifications for Consolidation

8.1 Proposed ops/lib/manifests/CORE.md

Core Context Manifest

Purpose

Defines the immutable kernel of truth required for all Stela operations.

Files

- PoT.md
- SoP.md
- TASK.md
- docs/MAP.md
- projects/README.md

8.2 Proposed ops/bin/synthesize Logic (Pseudocode)

Bash

```
#!/bin/bash
# ops/lib/scripts/synthesize.sh

function resolve_manifest() {
  local manifest="$1"
  # Logic to extract paths from markdown backticks
  # Logic to resolve "Inherits:..." directives
}
```

```
function check_hazards() {
  local path="$1"
  if [[ "$path" == "docs/library/"* ]]; then
    # Exception: Allow library ONLY if scope is 'discovery'
    if]; then
      echo "Error: Context Hazard detected in $path" >&2
      exit 1
    fi
  fi
}

function synthesize() {
  files=$(resolve_manifest "$MANIFEST")
  for file in $files; do
    check_hazards "$file"
    echo "===== FILE: $file ====="
    cat "$file"
  done
}
```

Works cited

1. OPEN-main-2dafc32f.txt
2. llms-txt: The /llms.txt file, accessed February 12, 2026, https://llmstxt.org/